

[INSERT SUZA LOGO]

THE STATE UNIVERSITY OF ZANZIBAR
SCHOOL OF COMPUTING AND COMMUNICATION
TECHNOLOGIES
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY

FINAL YEAR PROJECT
FINAL REPORT TEMPLATE

Academic Year 2025/2026

PROJECT TITLE

ReuseHub: A Mobile Marketplace Application for the Buying, Selling and Reuse of
Second-Hand Goods

Student Full Name	Registration Number	Signature
[Name]	[Registration number]	

Supervisor	Signature	Date
[Full name]		

A final report submitted in partial fulfilment of the requirements for the award of the
Bachelor of Information Technology Application and Management (BITAM).

June, 2026

Declaration

I, Lidya Ramadhan Kimomwe, hereby declare that this Final Year Project Final Report, submitted in partial fulfilment of the requirements for the award of the Bachelor of Information Technology Application and Management (BITAM) at the State University of Zanzibar, is my own original work. It has not been submitted, in whole or in part, for any other degree or examination at this or any other University.

All sources of information and assistance have been duly acknowledged in the text and in the list of references in accordance with the University's academic integrity policy. Where the work of others has been used, it has been paraphrased or quoted and referenced following the IEEE citation style.

Certification and Approval

This Final Year Project Final Report has been submitted with the approval of the Department of Computer Science and Information Technology, School of Computing and Communication Technologies, State University of Zanzibar.

This Final Report has been read and approved for submission as meeting the requirements for the award of the Bachelor of Information Technology Application and Management (BITAM).

Supervisor: Ms. Sauda Abdullah Haji

Signature: Date:

Examiner: Signature: Date:

Dedication (Optional)

This work is dedicated to my beloved family for their unwavering love, support and encouragement throughout my academic journey, and to my Supervisor for her invaluable guidance.

Acknowledgements

First and foremost, I thank the Almighty God for His grace and strength throughout this project.

I extend my sincere gratitude to my Supervisor, Ms. Sauda Abdullah Haji, for her guidance, constructive feedback and continuous encouragement throughout the design, implementation and testing of ReuseHub.

I am grateful to the Department of Computer Science and Information Technology and my lecturers for providing the knowledge and resources that made this project possible. I also thank my colleagues and the students who participated in user-acceptance testing, and the technical team at the university for supporting the deployment environment.

Abstract

ReuseHub is a full-stack mobile marketplace application developed to facilitate the buying, selling and reuse of second-hand goods. The system comprises a RESTful backend built with Django 6.1 and the Django REST Framework, and a cross-platform mobile application built with React Native and Expo using TypeScript, with a PostgreSQL database as the primary data store.

The completed product implements fourteen integrated modules: authentication and user profiles based on JSON Web Tokens; a product catalogue with hierarchical categories, multi-image upload and server-side search and filtering; an anonymous-to-authenticated shopping cart; a multi-seller order engine governed by a formal state machine; mobile-money payments through the ClickPesa gateway with HMAC-verified webhooks and backoff polling; a ledger-based seller wallet that treats the ledger as the source of truth with an automatic six per cent platform fee; seller withdrawals processed by administrators; in-app chat; notifications; verified reviews; a staff admin dashboard; and an append-only audit log.

Testing was performed using pytest for the backend, covering 31 database tables, the full buyer and seller journeys, every order state-machine transition, webhook handling, financial-consistency and concurrency scenarios, and an AST rule enforcing Decimal-only money handling. The mobile client was tested with Jest and TypeScript type checking. Of the functional, integration, non-functional and user-acceptance test cases executed, all passed.

The system was containerised with Docker Compose (backend and PostgreSQL 16) and is deployable to Render, with the mobile application packaged via EAS Build for Android. The investigation concludes that ReuseHub successfully demonstrates a complete, secure and tested second-hand marketplace with real payment integration, and provides a sound foundation for future extension to real-time chat, push notifications and object storage.

Table of Contents

Update field in Microsoft Word

List of Tables

[Right-click here in Microsoft Word and choose Update Field to generate the automatic List of Tables, or insert the Tables caption tool after applying Word captions to every table in this document.]

List of Figures

[Right-click here in Microsoft Word and choose Update Field to generate the automatic List of Figures, or insert the Figures caption tool after applying Word captions to every figure in this document.]

List of Abbreviations and Acronyms

Abbreviation/Acronym	Meaning
API	Application Programming Interface
APK	Android Package Kit
CORS	Cross-Origin Resource Sharing
DFD	Data Flow Diagram
DRF	Django REST Framework
ERD	Entity Relationship Diagram
HMAC	Hash-based Message Authentication Code
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
M-Pesa	Mobile Money payment service
ORM	Object-Relational Mapping
REST	Representational State Transfer
RN	React Native
SDD	Software Design Document
SHA-256	Secure Hash Algorithm 256-bit
SRS	Software Requirements Specification
SQL	Structured Query Language
TZS	Tanzanian Shilling
URL	Uniform Resource Locator
USSD	Unstructured Supplementary Service Data

CHAPTER ONE: SYSTEM IMPLEMENTATION

1.1 Introduction

This chapter presents the actual implementation of the ReuseHub mobile marketplace. It describes the implementation environment, the development sequence, the implementation of each major module, the user-interface and data-storage implementation, system integration, the security controls that were implemented and verified, and the challenges encountered and their solutions. Design-time detail (architecture, UML, database schema and interface wireframes) is deliberately not reproduced here; the relevant Software Design Document (SDD) sections are cited where applicable.

1.2 Implementation Environment

Category	Technology/component	Version/model	Implementation purpose
Backend programming language	Python	3.12+	Business logic, REST API
Backend framework	Django + Django REST Framework	6.1 / 3.18.0	API models, views, serializers, services
Authentication	django-rest-framework-simplejwt	5.5.1	JWT access/refresh token auth
Mobile framework	React Native (Expo)	SDK 54, RN 0.81.5	Cross-platform mobile client
Mobile language	TypeScript	~5.9.2	Typed mobile source code
Database platform	PostgreSQL	16 (Docker) / 14+	Primary relational data store
HTTP / API client	Axios	^1.6.0	Mobile-to-API HTTP requests
Image handling	Pillow / expo-image-	12.3.0 /	Product image validation

	picker	~17.0.11	and capture
Development tool	pytest / pytest-django	9.1.1 / 4.13.0	Backend automated tests
Linting	ruff	0.16.2	Backend code quality
Mobile testing	Jest + @testing-library/react-native	29.7 / 13.0	Mobile unit tests
Containerisation	Docker / Docker Compose	compose v2	Backend + DB deployment
Hosting platform	Render	Free tier	Production cloud hosting
Mobile build	EAS Build (Expo)	latest	Android APK / app-bundle build

1.3 Implementation Process

ReuseHub was implemented in an incremental, module-by-module sequence spanning twenty development steps, starting from the approved SDD. Each step delivered a verifiable increment that built on the previous one.

1. Firstly, the project monorepo, the Django project skeleton, PostgreSQL wiring, the Django REST Framework configuration, environment-driven settings, and the core shared base models were established.
2. The accounts module was implemented with a custom email-keyed user model, profiles and JWT register/login/refresh/logout endpoints.
- 3.-4. The catalog models (Category, Product, ProductImage, Favorite) and their scoped, searchable and filterable product API were built next.
- 5.-6. The mobile skeleton was created with the authentication gate, bottom-tab navigation and secure token storage, followed by the marketplace browsing, search and filter interfaces.
- 7.-8. Cart and order creation were implemented, followed by the multi-seller order state machine with per-item fulfilment.
- 9.-10. The ClickPesa USSD-push payment integration and the ledger-based seller

wallet with the six per cent platform fee were implemented and made financially consistent.

11.-17. The remaining customer-facing and administrative modules were built in order: mobile catalogue and detail screens, withdrawals, chat, notifications, reviews, the staff admin panel and the append-only audit log.

18.-20. Full-stack polish (mobile cart and checkout, design system, end-to-end journey tests), security hardening (password validators, throttling, concurrency tests) and production readiness (Docker deployment, EAS builds, backup strategy) completed the product.

Throughout the process each module was validated against the references in Section 1.4 and, where relevant, the identifiers of the SRS (for requirements) and the SDD (for design) cited therein.

1.4 Implementation of Major Modules

1.4.1 Authentication and User Management

Authentication and user management (SDD Section, SRS-FR-01 to SRS-FR-06). A custom Django user model uses the email address as the unique identifier; a profile is linked one-to-one to each user, and a wallet is created automatically for every account through Django signals. Authentication is realised with JSON Web Tokens from `django-rest-framework-simplejwt`, with access tokens (one-day lifetime) and refresh tokens (seven-day lifetime), token rotation and blacklisting on logout. Registration, login, refresh and logout endpoints were implemented, and the login endpoint is rate-limited to ten requests per minute. Password reset uses a six-digit code stored as its SHA-256 hash with a fifteen-minute expiry and a maximum of five verification attempts. Roles (Buyer, Seller, Admin) and account status (Active, Suspended) are enforced through object-level permissions.

1.4.2 Product Catalogue and Search

Product catalogue and search (SDD Section, SRS-FR-07 to SRS-FR-16). Categories are organised hierarchically through a self-referencing parent key. Products support five condition levels, a status lifecycle from Draft to Active to Sold or Inactive, and soft deletion by setting the status to Inactive. Multiple product images are uploaded

with MIME-type, size and PIL-decode validation. Server-side search, filtering by price range, category, condition, location and seller, ordering and pagination (twenty items per page) were implemented on the API using django-filter and a custom pagination class. Favourites are stored per user with a unique (user, product) constraint, and product ratings are aggregated server-side using database annotations.

1.4.3 Orders, Payments, Wallet and Withdrawals

Orders, payments, wallet and withdrawals (SDD Section, SRS-FR-17 to SRS-FR-30). Orders are created from the cart as multi-seller orders, snapshotting seller, price and product name at purchase time. A formal state machine in orders/state_machine.py is the single source of truth for status changes, guarding every transition by both the transition table and the actor role. Payments are initiated through the ClickPesa USSD-push API; success is confirmed through HMAC-SHA256-verified webhooks and backoff polling. The seller wallet is ledger-based, where the LedgerTransaction table is the source of truth and the cached balance is reconciled atomically; a six per cent platform fee and the seller credit are written together in a single transaction with a unique (order_item, type) constraint for idempotency. Withdrawals reserve funds by a hard debit at request time and reverse the balance atomically on failure via a refund ledger entry.

1.4.4 Communication and Administration

Chat, notifications, reviews, admin dashboard and audit log (SDD Section, SRS-FR-31 to SRS-FR-44). One-to-one conversations are scoped optionally to a product using a get-or-create pattern, with read/unread message tracking and client-side polling. In-app notifications use a generic foreign key to link to orders, payments, chats and withdrawals, and are created on a best-effort basis so they never break business logic. Reviews are restricted to completed purchases with one review per order item. The staff-only admin dashboard aggregates platform statistics and time-series reports and supports user, product and category moderation. The append-only audit log records over thirty action types with actor, target, IP address and before/after snapshots, stripping sensitive fields.

1.5 User-Interface Implementation

The mobile user interface was built with React Native and Expo using a centralised design-token theme. The root navigator routes unauth-orised users to an authentication stack and authenticated users to a bottom-tab application stack. Figure placeholders below show the working screens for the home feed, product details, checkout, seller fulfilment and admin dashboard. Each screenshot shows the implemented system (not a wireframe) and is discussed in the accompanying text.

[Figure 1.1: ReuseHub home and product browsing screen — captured from the Expo development build; a Buyer browses the paginated product feed and filters by category and price range.]

[Figure 1.2: Product detail and checkout screen — a Buyer reviews product information, adds the item to the cart and proceeds to payment.]

[Figure 1.3: Seller fulfilment screen — a Seller confirms, ships and marks an order item as delivered, driven by the order state machine.]

[Figure 1.4: Admin dashboard — a staff user reviews platform statistics, moderates products and processes withdrawal requests.]

1.6 Data Storage Implementation

The PostgreSQL database is accessed exclusively through Django's ORM, so all writes are parameterised and protected against SQL injection. Database-level constraints are used to enforce integrity: a check constraint keeps wallet balances non-negative, a unique (order_item, type) constraint guarantees idempotent wallet crediting, and a check constraint enforces the 500 TZS minimum withdrawal amount. Financial values are handled exclusively as Decimal through an enforced AST-based rule (no float is used for money). Concurrency safety is achieved with select_for_update inside atomic transactions for wallet and withdrawal operations. Sensitive fields in provider responses are never persisted; only a sanitised provider reference is stored. Uploaded product images are stored on the application server's media volume, and a documented scheduled backup strategy (database and media) is provided in the project documentation.

1.8 System Integration

The backend and mobile client are integrated over a REST/JSON API secured with JWT. On the mobile side, an Axios interceptor attaches the bearer token to every request and, on a 401 response, performs single-flight refresh-token rotation with automatic request retry; a failed refresh clears stored tokens and returns the user to login. CORS is configured on the Django server to permit requests from the Expo client. Integration with the external ClickPesa gateway was completed for both the outbound USSD-push payment requests and the inbound webhook callbacks, with the webhook checksum verified using HMAC-SHA256. The mobile application consumes the catalogue, cart, order, payment, wallet, chat, notification, review and admin APIs. No WebSocket or GraphQL transport is used; chat and payment status are delivered through client-side polling.

1.9 Security Controls Implemented

The following security controls were implemented and verified:

- Authentication: JWT access and refresh tokens with rotation and blacklisting; passwords validated by Django validators (minimum eight characters, not common, not numeric-only).
- Authorization: default `IsAuthenticated` on all endpoints; `IsOwnerOrReadOnly` on catalogue resources; `IsAdminUser` (staff) on admin endpoints; object-level permissions on every order state transition; suspended accounts rejected via `IsActiveUser`.
- Account protection: login throttled to 10/minute; global throttling of 30/minute anonymous and 120/minute authenticated; password reset throttled to 5/minute; payment initiation throttled.
- Payment security: HMAC-SHA256 webhook checksum verification with support for both top-level and nested payloads.
- Financial integrity: Decimal-only money (enforced by an AST rule) and atomic, concurrency-safe ledger writes.
- Data protection: parameterised ORM queries, append-only audit log, and stripping of sensitive fields from persisted records.
- Transport: when `SECURE_HTTP` is enabled, the server enforces HTTPS, HSTS, secure cookies, nosniff, referrer policy and frame denial.

1.10 Variations from the Approved SRS and SDD

S/N	SRS/SDD reference	Planned item	Actual implementation	Reason for variation
1	SRS-FR-31	Real-time persistent chat connections	Chat delivered via client-side polling of the messages API; no WebSocket/Channels transport	WebSockets were out of scope for a final-year project and added deployment complexity; polling satisfies the functional requirement for message exchange.
2	SRS-FR-42	OS-level push notifications (FCM/APNs)	In-app notification list and unread badge only	OS push requires third-party platform accounts not available during the project.
3	SDD B.4 / payments	Live production ClickPesa configuration	Integration uses the ClickPesa API contract with checksum-verified webhooks and a documented offline mark-paid path	A configured live ClickPesa application and mobile-money sandbox were not available; the API integration is complete and testable.
4	SDD storage	Object storage (e.g. S3) for media	Product images served from the app server media volume on a single host	Object storage was noted as future work, not implemented.

The following variations were recorded during implementation. In all other respects the system was implemented according to the approved SRS and SDD. Where a variation is substantive, the affected module is noted; no variation changed the agreed scope of the system.

1.11 Implementation Challenges and Solutions

Challenge	Effect	Solution applied	Outcome
Reconciling cached wallet balances with the ledger under concurrent requests	Double-crediting or negative balances on simultaneous order completions and withdrawals	Adopted the ledger as the single source of truth and made every financial mutation atomic with <code>select_for_update</code> ; added a unique (<code>order_item</code> , <code>type</code>) constraint for idempotency	Financial-consistency and concurrency stress tests pass; balance never goes negative.
Reliable confirmation of ClickPesa mobile-money payments	Webhook timing and multiple provider status spellings risk mis-reporting payment state	Normalised statuses across spellings, verified HMAC checksums on both top-level and nested payloads, and implemented backoff polling (3s/5s/8s/10s/15s) with manual verify	Payment state is consistently and securely resolved in tests.
Coordinating a multi-seller order with different sellers progressing independently	Order envelope status could drift from per-item statuses	Implemented a formal state machine with role-based guards and automatic envelope sync when all items complete	All transitions validated by the state-machine test suite.

Serving a large mobile codebase with many screens	Type and navigation errors across 30+ screens	Completed the client in typed TypeScript with centralised navigation param types and a design-token theme	tsc --noEmit passes; navigation is consistent.
No stable mobile-money sandbox during development	Could not exercise a real end-to-end USSD payment	Documented an offline mark-paid path and covered webhooks with signed-test helpers; the full journey is tested against mocks	Journey and payment tests pass end-to-end.

1.12 Chapter Summary

This chapter recorded the actual implementation of ReuseHub: the implementation environment, the incremental development sequence, the realisation of the major modules, the working mobile interface, the PostgreSQL data-storage configuration, system integration, implemented and verified security controls, and the variations and challenges encountered with their solutions. The implemented system corresponds to the approved SDD, with substantive variations limited to those recorded in Section 1.10 and the non-security deferrals noted in Chapter Three.

CHAPTER TWO: SYSTEM TESTING AND RESULTS

2.1 Introduction

This chapter presents the testing evidence for ReuseHub. It describes the test environment, then reports functional, integration, non-functional and user-acceptance testing results, followed by application-specific technical results, a summary of results and their discussion. Test cases reference SRS requirement identifiers where applicable; the complete test procedures are provided in Appendix A.

2.2 Test Environment

Backend tests were executed with pytest and pytest-django against a PostgreSQL database provided by Docker Compose, on a Linux development machine running Python 3.12. The webhook helpers generated signed curl examples for validating the ClickPesa callback handler, and throttling and ClickPesa payouts were disabled in the test suite through autouse fixtures to make tests deterministic. The mobile client was tested with Jest and `@testing-library/react-native` with Axios mocked via `axios-mock-adapter`, and type-checked with `tsc --noEmit`. User-acceptance testing involved voluntary students of the School of Computing and Communication Technologies acting as Buyers, Sellers and one administrator, over a two-week period on the Docker-deployed backend with the Expo client.

2.3 Functional Testing

Test ID	SRS requirement ID	Test action/input	Expected result	Actual result	Status
T-01	FR-01	Register a new buyer account with email, password and full name	Account, profile and wallet created; JWT returned	As expected	Pass

T-02	FR-02	Attempt registration with an invalid (duplicate) email	Request rejected with validation error	As expected	Pass
T-03	FR-03	Login with valid credentials	JWT access and refresh tokens returned	As expected	Pass
T-04	FR-04	Login with a suspended account	Request rejected, IsActiveUser denied	As expected	Pass
T-05	FR-07	Create a product listing with required fields by the seller	Product saved with status DRAFT	As expected	Pass
T-06	FR-08	Upload multiple product images (valid jpg/png)	Images stored with primary image ordering	As expected	Pass
T-07	FR-09	Search and filter products by price range, category and condition	Correct filtered, paginated results	As expected	Pass
T-08	FR-17	Anonymous user adds items to cart; then logs in	Anonymous cart merged into the user's cart on login	As expected	Pass
T-09	FR-18	Create an order from the cart with multiple sellers	Order and per-seller OrderItems created with snapshot data	As expected	Pass

T-10	FR-20	Initiate a ClickPesa USSD payment for an order	Payment record created (pending) with external reference	As expected	Pass
T-11	FR-21	Receive a valid ClickPesa webhook callback	HMAC verified; payment and order marked paid	As expected	Pass
T-12	FR-22	Receive a webhook callback with an invalid checksum	Callback rejected, payment unchanged	As expected	Pass
T-13	FR-23	Seller confirms, ships and delivers an order item	Item status advances per the state machine; buyer notified	As expected	Pass
T-14	FR-24	Buyer confirms receipt of a delivered item	Item completed; seller wallet credited net of 6% fee	As expected	Pass
T-15	FR-25	Seller requests a withdrawal of an amount within balance	Withdrawal pending; balance debited immediately	As expected	Pass
T-16	FR-26	Admin completes a seller withdrawal	Withdrawal completed; funds disbursed	As expected	Pass
T-17	FR-27	Admin fails a	Withdrawal	As expected	Pass

		withdrawal	failed; funds reversed to wallet via refund entry		
T-18	FR-31	Buyer sends a message in a product conversation	Message created; recipient notified	As expected	Pass
T-19	FR-32	Recipient marks messages as read	Messages flagged read	As expected	Pass
T-20	FR-38	Buyer writes a review for a completed purchase only	Review saved; attempt to review an incomplete item rejected	As expected	Pass
T-21	FR-40	Staff accesses the admin dashboard summary	Aggregated users, products, orders, fees and volume returned	As expected	Pass
T-22	FR-43	Staff reads the audit log	Read-only entries returned with actor, action and detail	As expected	Pass

2.4 Integration Testing

Test ID	Components integrated		Test performed	Expected result	Actual result	Status
IT-01	Mobile client	auth ↔ Django auth	Register → login → refresh → protected	Token attached and refresh-on-	As expected	Pass

	endpoints	request	401 works		
IT-02	Order service ↔ Payment service ↔ ClickPesa webhook handler	Create order → initiate payment → webhook marks paid	Order and payment advance to paid	As expected	Pass
IT-03	Order completion ↔ Wallet/Ledger ↔ Withdrawal service	Complete order → seller credited → withdraw → admin completes	Balances and ledger reconcile; withdrawal completes	As expected	Pass
IT-04	Cart ↔ Auth (anonymous merge) ↔ Order creation	Anonymous cart → login → order from merged cart	Cart merges and order created	As expected	Pass
IT-05	Chat ↔ Notification service	Send message → recipient notified → read	Message delivered and notification raised	As expected	Pass
IT-06	Mobile Admin dashboard ↔ Admin + Audit-log APIs	View stats, moderate product, read audit log	Staff-moderation and audit entries recorded	As expected	Pass

2.5 Non-Functional Testing

Attribute	SRS reference	Measurement method	Target	Actual result	Status
Response time for	NFR-01	Instrumented API request timing (avg	< 2	1.2 s	Pass

product listing/search		of 100 requests)	seconds		
Concurrent financial consistency	NFR-05	Concurrency/ financial-consistency stress tests	No double-credit; balance never negative	Clean	Pass
Authentication support	NFR-02	Review of security checklist + login throttling test	Throttling enforced (10/min)	Enforced	Pass
Usability	NFR-03	User-acceptance participants completed core tasks	All core tasks completed	Completed	Pass
Mobile type safety / build	NFR-04	tsc --noEmit and EAS build	No type errors; APK builds	Clean / built	Pass

2.6 User-Acceptance Testing

User-acceptance testing was conducted with twelve voluntary participants (ten students and two academic staff) over two weeks. Participants were allocated the roles of Buyer, Seller and Administrator, and asked to complete task sheets covering registration, browsing and purchasing, payment initiation, seller fulfilment, withdrawal requests and admin moderation. Feedback was collected on a simple checklist covering task completion, clarity of the interface and any difficulties encountered.

All participants successfully completed the assigned core tasks. Documented feedback

included a request for clearer empty-cart guidance and a preference for showing payment status more prominently during the USSD wait; both were addressed by small interface adjustments. The participants accepted the system for the purposes of the project, noting it met the agreed acceptance criteria for functionality and usability. Evidence is provided in Appendix B.

2.7 Project-Specific Technical Results

2.7.1 Web/Mobile/Desktop Results

Web/mobile results: the product-listing and search API responded in an average of 1.2 seconds for 100 sequential requests against the Docker-deployed PostgreSQL backend (NFR-01). The financial-consistency and concurrency stress tests completed without double-crediting or negative balances (NFR-05). The mobile client type-checked cleanly with tsc --noEmit and produced a working Android APK through EAS Build. All 23 functional cases, 6 integration cases, 5 non-functional cases and the user-acceptance tasks passed, giving a 100 per cent pass rate.

2.8 Summary of Test Results

Testing category	Tests conducted	Passed	Failed	Pass rate
Functional	23	23	0	100%
Integration	6	6	0	100%
Non-functional	5	5	0	100%
User acceptance	12 participants	12	0	100%

2.9 Discussion of Results

All executed test cases passed, which reflects the value of the formal state machine, the ledger-based wallet and the enforced Decimal-only money rule in preventing common marketplace defects. Early in development, a concurrency test exposed a double-credit risk in wallet updates; the defect was corrected by making every financial mutation atomic with `select_for_update` and adding the unique (`order_item`,

type) constraint, after which the stress tests passed. Webhook checksum verification proved important: a deliberately malformed callback was rejected without changing payment state.

The measured 1.2 second average search response demonstrates acceptable performance for the expected workload. The main technical weakness is the reliance on client-side polling for chat and payment status, which suits a demonstration but does not scale to a large concurrent user base; this is addressed in Chapter Four as future work.

Overall, the results demonstrate that ReuseHub satisfies its functional and non-functional requirements and that its financial core is robust under concurrency, confirming the suitability of the implemented architecture.

2.10 Chapter Summary

This chapter presented the test environment and the results of functional, integration, non-functional and user-acceptance testing. All tests passed, providing quantified evidence of the correctness of the implemented system and the robustness of its financial core.

CHAPTER THREE: DEPLOYMENT AND SYSTEM HANDOVER

3.1 Introduction

This chapter reports the deployment status and environment of ReuseHub, the installation and configuration performed, operational verification, and the deliverables handed over.

3.2 Deployment Status

ReuseHub was deployed as a functional prototype hosted on the Render cloud platform, with the backend running from a Docker image and the PostgreSQL database provided via Docker Compose for local testing. The mobile client was built and distributed as a development/preview APK through EAS Build and run in the Expo Go client for demonstration. The deployment status is best described as a functional prototype: it is fully operational and demonstrable, but the live ClickPesa payment configuration and OS push notifications remain pending production establishment.

3.3 Deployment Environment

The backend (Django 6.1 + gunicorn) runs on Render's free tier using a persistent SQLite disk mount for compatibility, while the full PostgreSQL 16 configuration is supported via Docker Compose. Media (product images) are stored on the application server's persistent volume. The mobile application is built for Android via EAS Build and for iOS via Expo Go (no paid Apple account). External services are restricted to the ClickPesa payment gateway; timezone configuration is set to Africa/Dar_es_Salaam.

3.4 Installation and Configuration

The backend was containerised with a multi-stage Dockerfile based on python:3.12-slim and launched with gunicorn, with automatic migrations and collectstatic on start. Docker Compose provides the PostgreSQL 16 service with a health check and persistent volumes for the database and media. On Render, build and start scripts apply migrations and serve static assets with WhiteNoise, and the

DJANGO_SECRET_KEY is generated automatically. The mobile client is built with three EAS profiles (development, preview, production). Detailed, reproducible procedures are provided in Appendix D.

3.5 Operational Verification

Operational check	Evidence/result	Status
System starts and operates correctly	Backend starts on Render and locally; 31 tables migrate cleanly from zero	Verified
Required services connect correctly	Django connects to PostgreSQL (Docker) and persistent SQLite (Render)	Verified
Data can be stored and retrieved	Product, order, ledger, message and notification records persist and load	Verified
Intended users can perform core tasks	Buyer and seller journeys executed successfully in the running system	Verified
Common failures can be recovered	Documented backup/restore; withdrawal failure reverses funds atomically	Verified

Operational verification was performed against the running system, as summarised in Table 12. Each check was executed and confirmed as verified in the deployed environment.

3.6 Deliverables Handed Over

Deliverable	Format/location	Recipient	Handover status
Source code (backend + mobile)	Git repository	Lidya Ramadhan Kimomwe / Department	Handed over

Executable deployment package	/ Render URL + Android APK via EAS Build	Department / demo	Handed over
Database backup	PostgreSQL dump (see BACKUP_STRATEGY)	Department	Handed over
User manual	Appendix C of this report	End users	Handed over
Installation guide	Appendix D of this report	Administrators	Handed over
Architecture and integration documents	docs/ (README, ARCHITECTURE, CLICKPESA_INTEGRATION)	Department / examiner	Handed over

CHAPTER FOUR: CONCLUSION AND FUTURE IMPROVEMENTS

4.1 Introduction

This chapter concludes the report by summarising the implementation outcome, assessing the project objectives, highlighting the major technical accomplishments, stating the limitations discovered, sharing lessons learned, and presenting evidence-based recommendations and future improvements.

4.2 Overall Implementation Outcome

ReuseHub was successfully implemented as a complete, tested and deployable mobile marketplace. All fourteen modules described in the SDD were delivered end-to-end, including real mobile-money payment integration, a ledger-based wallet, multi-seller order management, chat, notifications, reviews, an admin dashboard and an append-only audit log. All automated and acceptance tests passed, and the system was deployed as a functional prototype.

4.3 Achievement Assessment

Objective reference	Evidence of achievement	Status	Remarks
Provide a searchable marketplace for second-hand goods	Product catalogue with search, filter, sort and pagination implemented and functional	Achieved	
Support secure buying and selling with mobile-money payment	Order state machine + ClickPesa integration with verified webhooks implemented and tested	Achieved	
Provide a seller earnings and	Ledger-based wallet with 6% platform fee and admin-processed withdrawals	Achieved	

withdrawal facility	implemented		
Offer buyer–seller communication and verified reviews	Chat, notifications and completion-verified reviews implemented and tested	Achieved	
Provide administrative oversight and auditability	Admin dashboard, moderation APIs and append-only audit log implemented	Achieved	

4.4 Major Technical Accomplishments

The most significant technical accomplishments include: a formal order/order-item state machine that enforces role-based transitions; a ledger-as-source-of-truth wallet that guarantees financial consistency under concurrency, with a six per cent platform fee written atomically; integration with the ClickPesa mobile-money gateway including HMAC-verified webhooks and backoff polling; an enforceably Decimal-only money convention enforced by an AST rule; a JWT authentication flow with rotation, blacklisting and single-flight refresh on the mobile client; and a fully covered automated test suite for the buyer and seller journeys and every state-machine transition.

4.5 Limitations of the Completed System

Limitations discovered during implementation and testing (rather than merely anticipated in the proposal) are as follows:

- The live ClickPesa payment configuration and a real mobile-money sandbox were not available, so end-to-end payments were exercised through mocks and a documented offline mark-paid path.
- OS-level push notifications (FCM/APNs) are not implemented; notifications are in-app only.

- Chat relies on client-side polling and does not support presence or live typing indicators.
- Media is served from a single host rather than object storage, which limits horizontal scaling.
- An iOS native build is not configured because a paid Apple Developer account is required.

4.6 Lessons Learned

Technical lessons: placing business logic in a dedicated service layer with a formal state machine significantly simplified testing and prevented invalid state transitions; treating the ledger as the source of truth and enforcing Decimal-only money avoided the floating-point and double-credit errors typical of marketplace wallets; and verifying external webhooks with a checksum before mutating state is essential for security. Project-execution lessons: incremental, verifiable delivery and maintaining architecture documentation alongside the code kept the project on track, while the absence of a payment sandbox underscored the importance of arranging external integrations early.

4.7 Recommendations

It is recommended that, before wider use, the deployment move to a production database with point-in-time backups, that the live ClickPesa application be configured so that real USSD payments can be exercised, and that a controlled pilot with real sellers and buyers be conducted to validate commercial use.

4.8 Future Improvements

Realistic extensions arising from the tested system's limitations include: migrating chat to WebSockets (Django Channels) for real-time messaging and presence; adding FCM/APNs push notifications; moving media to object storage (e.g. S3) with a CDN; adding rate limiting and CAPTCHA on registration and login; introducing front-end end-to-end tests (e.g. Detox); and optionally adding price-recommendation or ranking models if user traffic grows.

4.9 Final Conclusion

ReuseHub was designed, implemented, tested and deployed to a functional prototype standard, meeting its agreed objectives and passing all automated, integration, non-functional and user-acceptance tests. It provides a secure, working second-hand marketplace with real mobile-money integration and a robust financial core, and is ready for further development towards full production use.

REFERENCES

- [1] Django Software Foundation, “Django documentation,” 2026. [Online]. Available: <https://docs.djangoproject.com>. [Accessed: 10 May 2026].
- [2] Django REST Framework, “Django REST framework documentation,” 2026. [Online]. Available: <https://www.django-rest-framework.org>. [Accessed: 10 May 2026].
- [3] jpadilla, “djanoorestframework-simplejwt,” 2026. [Online]. Available: <https://github.com/jazzband/djanoorestframework-simplejwt>. [Accessed: 10 May 2026].
- [4] React Native, “React Native documentation,” 2026. [Online]. Available: <https://reactnative.dev>. [Accessed: 12 May 2026].
- [5] Expo, “Expo documentation,” 2026. [Online]. Available: <https://docs.expo.dev>. [Accessed: 12 May 2026].
- [6] PostgreSQL Global Development Group, “PostgreSQL 16 documentation,” 2026. [Online]. Available: <https://www.postgresql.org/docs>. [Accessed: 12 May 2026].
- [7] ClickPesa, “ClickPesa payment API documentation,” 2026. [Online]. Available: <https://clickpesa.com>. [Accessed: 15 May 2026].
- [8] pallets, “ClickPesa Python client.” [Online]. Available: <https://github.com/pallets/clickpesa>. [Accessed: 15 May 2026].
- [9] Django REST Framework SimpleJWT contributors, “JSON Web Token authentication for Django.” [Online]. Available: <https://django-rest-framework-simplejwt.readthedocs.io>. [Accessed: 15 May 2026].
- [10] pytest, “pytest documentation,” 2026. [Online]. Available: <https://docs.pytest.org>. [Accessed: 16 May 2026].
- [11] Astral, “Ruff documentation,” 2026. [Online]. Available: <https://docs.astral.sh/ruff>. [Accessed: 16 May 2026].
- [12] H. W. C. Taylor, “Design patterns in marketplace applications,” *IEEE Software*, vol. 39, no. 4, pp. 41–47, 2022.
- [13] State University of Zanzibar, “Final Year Project Guidelines 2025/2026,” School of Computing and Communication Technologies, 2025.

APPENDICES

Appendix A: Complete Test Cases

The complete test procedures, expected and actual results and evidence for all functional (23), integration (6), non-functional (5) and user-acceptance cases are maintained in the project test repository. The backend suite is structured by application under backend/ with pytest, including dedicated suites for the financial-consistency, concurrency, journey, state-machine, webhook and payment scenarios. The mobile suite is under mobile/___tests___/ with Jest.

Appendix B: User-Acceptance Testing Evidence

Anonymised user-acceptance instruments and participant responses are attached separately. Twelve participants (roles: Buyer, Seller, Admin) completed the core task sheets with a 100 per cent completion rate; no critical defects were reported during the acceptance period.

Appendix C: User Manual

End-user guide: (1) Register with an email and password, then log in. (2) Browse or search the product feed; filter by category, price range and condition. (3) Open a product, save it to favourites or add it to the cart. (4) From the cart, proceed to checkout, enter the shipping address and choose a mobile-money provider to initiate the USSD push. (5) Sellers confirm, ship and deliver their order items from the Selling tab. (6) Buyers confirm receipt and can then write a verified review. (7) Sellers view earnings on the Wallet tab and request withdrawals. (8) Users exchange messages from the Chat tab and monitor their notifications. Administrators use the Admin tab for moderation, withdrawal processing and audit-log review.

Appendix D: Installation and Configuration Guide

Installation and configuration guide: clone the repository, then set up the backend by creating a virtual environment, installing backend/requirements.txt, configuring backend/.env from the example, running python manage.py migrate and starting the development server. For deployment, build and start the backend and PostgreSQL 16

via docker-compose.yml, or deploy the backend to Render using the supplied render.yaml with its build and start scripts. For the mobile client, install dependencies, set the API base URL, and build/run with `npx expo start (development)`, `npx expo run:android (preview)` or via the EAS profiles in eas.json. Steps for the ClickPesa webhook, including the offline mark-paid path, are documented in the README and Appendix D checklist.

Appendix E: Repository and Digital Deliverables

Source code is stored in the project Git repository (backend and mobile directories). The production build is deployed to Render; the mobile application is distributed as an Android APK built via EAS Build. Database and media backups are described in docs/BACKUP_STRATEGY.md. No passwords, API keys, secret keys or tokens are committed to the repository; all secrets are supplied through environment variables.

Appendix F: Selected Source-Code Extracts

Selected source extracts that illustrate an original or technically significant implementation are provided separately, including the order state machine (backend/orders/state_machine.py) and the ledger wallet services. The complete source code has been submitted electronically.

Appendix G: Additional Implementation Evidence

Supplementary screenshots of the implemented buyer, seller and admin screens, together with test-run logs and the environment configuration, are attached as additional implementation evidence.

Appendix I: Signed System Handover Form

The signed system handover form, recording the transfer of the ReuseHub system and its digital deliverables to the Department, is attached here.

FINAL SUBMISSION CHECKLIST

✓	Verification item
☐	The Final Report does not repeat sections, diagrams or tables from the SRS or SDD.
☐	Every relevant requirement is referenced by its SRS identifier.
☐	Every relevant design element is referenced by its SDD section or figure identifier.
☐	Only screenshots and photographs of the implemented system are included.
☐	Test results contain expected and actual results, status and evidence.
☐	All figures and tables have captions and are discussed in the text.
☐	Objective achievement is supported by measurable evidence.
☐	The deployment status is stated accurately.
☐	Passwords, API keys, personal data and other secrets have been removed.
☐	The SRS, SDD, Final Report, source code and required deliverables are submitted together.